Module 2: Database Models and SQL

1. Database Security

- Protect DB from unauthorized access, misuse or malicious attack
- Maintain confidentiality integrity and availability of data

i. **Access control**: Only authorized users can access; permissions and roles
ii. **Encryption**: Unreadable if stolen, uses protocols
iii. **Backups and Recovery**: Restore if corrupted/deleted
iv. **DB Firewalls**: Protective barrier between DB and external threats
v. **Patch Management**: Regular updates and scanning for threats

2. Entity-Relationship (ER) model

- Represents relationships between data entities in structured and systematic way.
- Easier to design and visualize DB before implementing in DBMS
- Key components:

i. Entity
- Real world object, concept or thing with distinct existence
- Has set of attributes that describe its properties
- Notation: Rectangle

ii. Attribute:
- Property/characteristic of entity
- Notation: Oval (connected to its entity)

iii. Primary Key
- Unique identifier that distinguishes record in entity

iv. Relationship
- Represents association between 2 or more entities (now related to each other)
- Notation: Diamond (connecting entities involved)

v. Cardinality
- No. of instances of 1 entity associated with each instance of them
- Notation: no. or symbols on lines (1 to 1, many to 1, many to many)

vi. Weak entity
- Can’t uniquely identify, relies on owner
- Notation: Double boxed rectangle

vii. Composite attribute
- Attribute with sub parts

viii. Multivalued attribute
- Multiple values
- Notation: Double rounded oval

3. Constraints

- Rules applied to data in tables to ensure accuracy, consistency and integrity
- Restrict type of data that can be inserted, updated, deleted

i. Primary Key constraint

- Each row in table has unique identifier
- No duplicate or NULL values allowed in PK columns
- E.g.: student ID → unique, each student has unique one

ii. Foreign Key constraint

- Creates link between 2 tables
- Ensures value in table 1 corresponds to valid value in table 2
- Primary key of another table

iii. Unique constraint

- Allows NULL values
- Ensures all values in column are unique across rows
- E.g.: Phone number

iv. Not Null constraint

- Ensures column doesn't have NULL values
- Every row must contain value for that column
- E.g.: student name (every student must have a name)

v. Check constraint

- Ensures all values in column satisfy a specific condition
- E.g.: price > 0 (price will always be > 0)

vi. Default constraint

- Restricts data to permissible values for a column
- Ensures data is used only from predefined domain (Set of values)
- E.g.: Gender (male/female/other)

vii. Referential integrity constant

- Ensures relationships between tables remain constant
- When FK used, ensures that data in FK matches data in referenced PK
- Cascading actions (automatically delete/update)

4. Aggregate functions (select sum(salary) from emp) ← example use

- Perform calculations on set of values and return single value
- Summarize data and provide insights

i. Count():

- Counts no. of rows in result set, no. of set of non-NULL values in column

ii. Sum():

- Adds up all values in numeric columns and returns total

iii. Avg():

- Calculates average of values in numeric columns

iv. Min():

- Returns smallest value in specific column

v. Max():

- Returns largest value in specific column

5. Views

i. **Virtual Table**: behaves like table when queried but doesn't store data on its own
the data view is fetched from 1 or multiple existing tables

ii. **Query Simplification**: Easier to reuse complex queries

iii. **Data Security**: restricts access to specific columns/rows of table control what data diff users can see

- **SYNTAX**
  - Create view VIEW-NAME as
  - Select COLUMN1, COLUMN2, ….
  - From TABLE1, TABLE2, ….
  - Where CONDITION ___ ;

	Benefits	Limitations
1	Simplified querying	Performance
2	Data Security	Updatability
3	Data Abstraction	Dependency
4	Reusability	

6. Mapping Cardinality

- Defines relationship between 2 entities in terms of no. of instances of 1 entity that can be associated with no. of instances of another entity
- There are 4 types:

i. One to one

- Each instance of 1 entity is associated with single instance of another entity and vice versa
- E.g.: Employee and Passport

ii. One to many

- Single instance of 1 entity is associated with multiple instances of another entity. However, each instance of 2nd entity is associated with 1 instance of 1st
- E.g.: Dept. has many employees, each employee related to one department

iii. Many to one

- Multiple instances of 1 entity associated with 1 instance of another entity
- E.g.: Many students assigned to 1 proctor, each student 1 proctor

iv. Many to many

- Multiple instances of 1 entity associated with multiple instances of other
- E.g.: Students enrol in multiple courses; each course has multiple students.